MoEBench: Accelerating System Benchmarking with a Robust Machine Learning Pipeline

ABSTRACT

Benchmark suites are vital for evaluating computing platforms, but their comprehensive, all-or-nothing execution model incurs prohibitive time and resource costs. While replacing physical execution with machine learning (ML) predictions is a tempting alternative, applying naïve ML directly to system benchmarking fails in practice due to highly dynamic runtime contention, inflexible run-to-completion workloads, and inherently imperfect production telemetry. To address these challenges, we present MoEBench, a lightweight benchmarking toolchain that reframes full-suite evaluation as a conditional selection and sparse reconstruction problem. Inspired by the Mixture-of-Experts principle, MoEBench utilizes a dynamic router to select a minimal, highly informative subset of test components based on real-time system context. Instead of running these selected tests to completion, it employs time-bounded probing to extract prefix-level execution signatures. A multi-output reconstructor then recovers the complete benchmark profile from these sparse observations. Crucially, MoEBench is engineered for real-world robustness; it embraces execution anomalies (e.g., timeouts, missing telemetry) as informative features and leverages uncertainty estimation to actively refine predictions during long-tail system states. Extensive evaluations across heterogeneous hosts demonstrate that MoEBench accurately recovers aggregate suite scores and subtest profiles while slashing wall-clock execution time by an order of magnitude.

CCS CONCEPTS

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

KEYWORDS

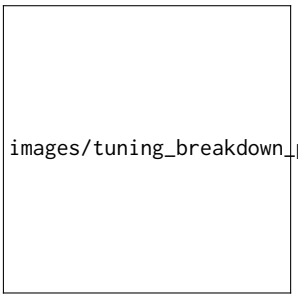
Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

. 2018. MoEBench: Accelerating System Benchmarking with a Robust Machine Learning Pipeline. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX



(a) Time breakdown of two representative KeenTune tasks.

Machine	UnixBench	PTS CPU
2U8G	1679 s	5424 s
4U8G	1677 s	26667 s
4U16G	1676 s	31756 s
8U8G	1678 s	1930 s
32U128G	1701s	15414s

(b) Full-suite benchmark execution time across machines.

Figure 1: Execution-time cost of benchmark-driven tuning and full-suite benchmark evaluation.

1 INTRODUCTION

Benchmark suites [**? ? ?**] characterize modern computing platforms by running standardized workloads across processors, memory, storage, system software, and accelerators to produce a repeatable performance profile beyond any single measurement. Such suites are widely used in performance-critical infrastructure workflows, including cloud instance comparison [**? ? ? ? ? ? ?**], monitoring [**?**], and performance autotuning [**? ? ? ?**]. However, the same breadth that makes benchmark suites useful also makes them costly to execute in full. For instance, as illustrated in Figure 1, benchmark evaluation can easily dominate the overall time budget of an autotuning task (Figure 1a) and incur substantial wall-clock overhead across different machine configurations (Figure 1b). Comprehensive suites often contain many heterogeneous workloads, each consuming wall-clock time and contending for CPU cores, memory bandwidth, storage bandwidth, or accelerator resources. While this cost may be acceptable for occasional offline evaluation, it becomes difficult to amortize when benchmarking must be repeated across many machines, instance types, software revisions, configurations, or time intervals. In deployed or shared environments, full-suite execution can also perturb the system being measured by competing for resources, interfering with co-located workloads, or changing thermal and frequency states. Moreover, conventional suites largely follow an all-or-nothing execution model: they run a fixed set of workloads and produce meaningful results only after the full run completes.

Because the system itself cannot magically compress the execution time of standard workloads, various approaches have attempted to mitigate this overhead. "Static-subset" approaches manually cherry-pick a fixed group of subtests, but they face a high barrier to generalization: a subset chosen for a CPU-bound instance falls short on an I/O-heavy machine [**?**]. In the middle ground, "zero-shot" machine learning techniques attempt to bypass execution entirely by predicting scores based on static host configurations [**? ?**]. However, they are fundamentally blind to transient runtime noise,

such as LLC thrashing or background I/O contention, causing them to produce overconfident yet wildly inaccurate predictions. Among all alternatives, the most reliable solution remains running the full suite to completion, but it pays the unacceptable cost of execution delays and severe resource interference in shared environments.

We take a new approach: let the benchmark be the benchmark and do not permanently truncate its workloads, but dynamically learn how to observe its minimal signature. The key to our approach is learning-driven sparse execution. Can we learn the behavior of a target system by running only a tiny fraction of the suite, and use the results to reconstruct the full performance profile with high accuracy? We introduce MoEBench, a lightweight benchmarking toolchain that reframes full-suite evaluation as a conditional selection and sparse reconstruction problem. We show how MoEBench helps tuning frameworks and monitoring infrastructures achieve full-suite visibility with an order of magnitude less wall-clock execution time.

The biggest challenge for MoEBench is to be as accurate as the traditional run-to-completion approach while operating in a highly unpredictable production environment. The key to avoiding massive time overhead is to dynamically schedule only the most informative subtests and truncate them early. However, keeping the evaluation reliable is a fundamental constraint. We need to handle incomplete execution traces, missing telemetry, and anomalous system states without letting the ML pipeline crash. To address this, MoEBench introduces three technical contributions.

First, MoEBench converts the rigid full-suite execution into a dynamic Mixture-of-Experts (MoE) routing problem. We take advantage of the fact that many benchmark components inherently stress overlapping hardware subsystems. In this view, executing every subtest is overkill. With this intuition, MoEBench employs a lightweight router that monitors the real-time system context (e.g., dynamic load, IPC) and computes a conditional Top-K subset of experts tailored to expose the current system's bottlenecks.

Second, with the selected subset, MoEBench employs time-bounded probing rather than run-to-completion execution. Many benchmark components are iterative rate tests where key performance metrics stabilize well before the official timeout. MoEBench enforces a strict, short execution window to capture a prefix-level signature. Crucially, the system is engineered to embrace real-world fragility: if an I/O probe times out due to heavy contention or eBPF tracers are blocked by hypervisors, MoEBench does not discard these as statistical anomalies. Instead, it encodes these execution failures as valid state features, ensuring robust inference without requiring perfectly clean traces.

Third, MoEBench balances the sparsity of observation with the demand for full-profile reconstruction via uncertainty estimation. While executing a sparse subset is fast, downstream tasks still expect a full-suite aggregate score. MoEBench utilizes a multi-output reconstructor to bridge this semantic gap. More importantly, to prevent the ML model from making blindly confident predictions on rare, long-tail machine states, we integrate a confidence-aware refinement loop. If the model's predicted variance exceeds a safe threshold, it dynamically triggers active refinement to probe additional components, ensuring that extreme performance anomalies are physically verified rather than incorrectly guessed.

2 BACKGROUND AND MOTIVATION

2.1 Benchmark Suites

Benchmark suites provide reproducible and quantifiable measurements for comparing system performance across machines, configurations, and software versions. Rather than relying on a single workload, a suite decomposes system performance into multiple components, each targeting a specific workload pattern or subsystem, such as computation, memory, storage, networking, process control, or inter-process communication. We denote a suite as

$$\mathcal{B} = \{b_1, b_2, \ldots, b_N\},$$

where b_i is the i-th component.

For a system state s, the full result of component b_i is

$$y_i = f_i(s, T_i, \epsilon_i),$$

where T_i is the execution budget required by the original benchmark protocol and ϵ_i denotes measurement noise. A full-suite run produces

$$\mathbf{y} = [y_1, y_2, \ldots, y_N],$$

and, when defined by the benchmark framework, an aggregate score

$$Y = A(\mathbf{y}).$$

The cost of full-suite benchmarking comes from two sources: the number of components and the duration of each component. Formally,

$$C_{\text{full}} = \sum_{i=1}^{N} C_i(T_i).$$

This cost can be substantial for suites with many components or long measurement intervals. However, not all observations are equally informative. Components may stress overlapping subsystems and therefore exhibit correlated results, while short executions may already expose the rate or latency trend of a full run. This motivates our design goal: estimate the full component vector and aggregate score using only sparse component observations and short execution windows.

2.2 Motivation

Insight 1: Sub-benchmarks Exhibit High Subsystem Redundancy. Although comprehensive suites contain dozens of subtests, many of these workloads stress overlapping hardware subsystems. Our profiling reveals that seemingly diverse tasks within UnixBench—such as integer computation (`dhry2reg`), process creation (`spawn`), and inter-process communication (`pipe`)—often bottleneck on the same underlying OS scheduling overheads or memory hierarchy mechanisms. Because these subtests are merely different software projections of the same underlying hardware capacity, executing all of them yields diminishing returns in information gain. This physical redundancy indicates that a carefully routed, minimal subset of experts is theoretically sufficient to expose the current system's overall performance limits.

Insight 2: Performance Metrics Exhibit Early Convergence. When full-suite execution is unavoidable, traditional methodologies mandate a rigid run-to-completion model. However, we observe that many benchmark components are fundamentally iterative rate tests. For instance, the Dhrystone and Whetstone components in

UnixBench [**?**] continuously execute integer and floating-point kernels over a prolonged loop. Our telemetry shows that the computed operation rate (e.g., operations per second) typically stabilizes and converges within the first few seconds of execution. Continuing the workload to its official timeout primarily reduces statistical variance rather than altering the fundamental performance metric. This insight reveals a massive opportunity for vertical cost reduction via aggressive, time-bounded probing.

Insight 3: Production Telemetry is Inherently Fragile. While the previous insights justify utilizing ML to extrapolate full scores from short, sparse probes, deploying such an ML pipeline in production reveals a fatal flaw: real-world execution is messy. In latency-critical or heavily contented cloud environments, short probes frequently timeout before returning valid application-level metrics. Furthermore, high-fidelity monitoring tools (such as eBPF or PMU performance counters) are routinely restricted by cloud hypervisors for security isolation. A standard ML pipeline treats these execution anomalies and missing feature dimensions as statistical outliers, abruptly crashing or discarding the data. This fragility underscores that an industrial-grade benchmarking toolchain must be engineered to treat timeouts and missing traces as informative system states, and must quantify prediction uncertainty to prevent wildly inaccurate estimations on out-of-distribution hardware.

3 METHODS

3.1 Overview

MoEBench approximates a complete benchmark run by observing only a small subset of informative sub-benchmarks. Given a target system, it first collects a system feature vector x, which captures static platform properties and lightweight runtime state. A router then ranks the benchmark subtests and selects a small subset for observation (Section 3.3). Each selected subtest is evaluated through a fixed-duration probe rather than a full official run (Section 3.4). The probe-predicted scores are encoded as sparse score-time pairs and passed to a reconstructor, which predicts all subtest scores and the aggregate suite score (Section 3.5). This *router* $\rightarrow$ *probe* $\rightarrow$ *reconstruct* pipeline combines conditional subset selection with time-bounded per-component estimation, achieving lower wall-clock cost than either partial full execution or probing every component.

MoEBench separates offline training from online evaluation. During offline training, complete benchmark runs provide supervision for all model components. The router learns to identify informative subtests, the probe estimator learns to predict full subtest scores from short-window telemetry, and the reconstructor learns to recover full-suite results from partial observations. During online evaluation, the trained models are reused on a new target system, so MoEBench only needs to profile the system and observe the selected subtests instead of running the entire benchmark suite.

3.2 Feature Collection and Engineering

Feature collection provides the contextual information required to reduce benchmark cost while preserving the ability to estimate full-suite performance. The framework observes three factors that affect measured performance: what the machine is, what state the machine is in, and how a benchmark behaves during its initial execution prefix.

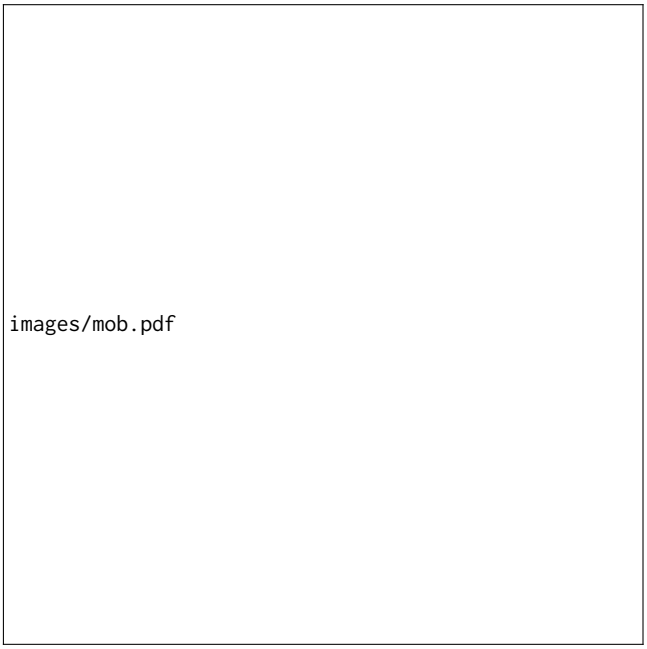


Figure 2: Overview of the MoEBench architecture. (1) Host OS: Collector aggregates static and dynamic metrics into system context. (2) Router: Router evaluates this context to select sparse benchmark subsets. (3) Probing: Selected cases undergo time-bounded probing yielding partial telemetry. (4) Reconstructor: Regressor predicts full profile from the encoded sparse telemetry.

For the i-th benchmark run, we define the system context vector as

$$\mathbf{x}_i = [\mathbf{x}_i^{\text{static}}, \mathbf{x}_i^{\text{dynamic}}]$$

where $\mathbf{x}_i^{\text{static}}$ describes stable host properties (e.g., CPU topology, total memory, OS versions) and $\mathbf{x}_i^{\text{dynamic}}$ captures the transient runtime state before benchmark execution (e.g., system load, hardware counters). The vector $\mathbf{x}_i$ is collected once before a full-suite run and is used by both the benchmark router and the early-stopping estimator.

During short-run probing, the framework further collects a probe-specific vector $\mathbf{p}_{i,j}$ for benchmark component b_j. Unlike $\mathbf{x}_i$, which describes the global context of the i-th run, $\mathbf{p}_{i,j}$ summarizes the execution prefix of a specific component, capturing wall-clock time, context-switch rates, and categorical identifiers. Table 1 summarizes these feature groups by collection stage and purpose.

Feature Processing and Domain-Aware Scaling. Raw system telemetry cannot be directly fed into the models. Instead of applying global statistical normalization (e.g., MinMax or standard scaling) which obscures physical limits across heterogeneous machines, MoEBench employs domain-aware semantic scaling. Static hardware capacities (e.g., total CPU cores, memory sizes converted

Table 1: Purpose-oriented feature groups organized by collection stage. Pre-run signals form the system context $\mathbf{x}_i$, while in-run probe signals form the probe vector $\mathbf{p}_{i,j}$.

Stage	Signal type	Purpose	Examples
Pre-run	Static host profile	Identify stable host configuration.	CPU topology, NUMA layout, memory size, compiler versions
	System pressure	Capture transient pre-run load.	CPU/GPU utilization, iowait, load average, runnable tasks
	PMU / perf	Characterize microarchitectural behavior.	IPC, cycles/s, instructions/s, cache/branch miss ratios
In-run	Probe progress	Record observed benchmark progress.	Configured probing duration, elapsed wall-clock time
	eBPF-based tracing	Observe prefix-level kernel activity.	Scheduling-switch rate, syscall-entry rate, eBPF availability
	Resource usage	Summarize prefix resource usage.	User/system/idle CPU ratio, minor faults, major faults
	Workload identity	Provide workload semantic context.	CPU, memory, I/O, thread, syscall, GPU category
	Probe flags	Mark probe mode and failures.	Micro workload flag, real-component flag, timeout flag

to uniform units) preserve their original magnitudes to explicitly inform the model of the system's ceiling. Dynamic metrics are transformed into stable ratios or rates: transient CPU utilization and I/O wait times are mapped to percentage ratios, while monotonically increasing counters (e.g., page faults, context switches, PMU instructions) are converted to per-second rates. Categorical variables, such as workload types or probe flags, are handled via one-hot encoding. All missing values are zero-filled to maintain a deterministic tensor shape for inference.

Cost-Aware Collection and Fallbacks. Extracting high-fidelity telemetry, such as PMU (Performance Monitoring Unit) events or eBPF traces, incurs overhead and relies on specific kernel configurations. Rather than statistically pruning these informative features, MoEBench adopts an engineering-driven fallback strategy to balance extraction cost and prediction accuracy. If high-overhead `perf` counters are unavailable or blocked by the hypervisor, the collector safely degrades to standard `/proc` statistics. Similarly, if eBPF tracing is disabled or GPU telemetry is absent on CPU-only instances, the framework zero-masks the corresponding vector dimensions. This robust collection pipeline ensures that the benchmarking methodology remains lightweight and executable across strictly isolated cloud tenants or bare-metal machines without crashing the evaluation process.

3.3 Router: Conditional Expert Selection

A full benchmark suite contains inherent redundancy. Although each sub-benchmark targets a specific subsystem, their results are jointly determined by the underlying system state—such as CPU throughput, memory hierarchy, and scheduler behavior. For example, in UnixBench, `dhry2reg` and `whetstone-double` reflect computation, while `pipe` and `syscall` reflect OS-level overheads. Because these sub-benchmarks are different projections of the same system configuration, a carefully selected subset preserves sufficient information to estimate the behavior of the full suite.

However, the most informative subset is not statically fixed. A CPU-bound configuration may require different representative tests than an I/O-bound configuration. MoEBench formulates benchmark acceleration as a conditional selection problem: given the current system feature vector x, dynamically select a small subset of sub-benchmarks that provides the highest utility for reconstructing the full-suite result.

Mixture-of-Experts Formulation and Learning-to-Rank. We adopt a mixture-of-experts view, treating each independently executable sub-benchmark as a measurement expert within a fixed pool:

$$\mathcal{E} = \{e_1, e_2, \ldots, e_n\}$$

Each expert is represented as $e_i = (\texttt{test\_id}_i, r_i, c_i)$, where r_i is the observed score and c_i is its execution cost.

Crucially, MoEBench does not treat routing as a standard multi-class classification problem. The goal is not to assign a single "best" label to a system state, but to evaluate the conditional relevance of *all* available tests and select the Top-K subset. Given x, the router assigns a relevance score to each expert, $s_i = f_\theta(x, e_i)$, and converts these into selection probabilities:

$$p_i = \frac{\exp(s_i)}{\sum_{j=1}^{n} \exp(s_j)}$$

MoEBench then isolates the Top-K experts to execute:

$$\mathcal{S}_K(x) = \mathrm{TopK}_{e_i \in \mathcal{E}}\, p_i$$

Model Exploration and Lightweight Architecture. To capture the non-linear interactions between system features and benchmark behavior, MoEBench explores three lightweight routing architectures, intentionally bypassing heavy Large Language Models (LLMs) or exhaustive AutoML searches to ensure inference latency remains negligible compared to physical execution times:

- **Gradient-Boosted Ranker (LightGBM):** Optimized directly for ranking using LambdaRank (optimizing NDCG), transforming expert relevance into a per-query rank.
- **Multi-Layer Perceptron (MLP):** A streamlined 2-layer network trained with pointwise MSE to predict individual expert relevance.
- **Graph Neural Network (GNN):** Utilizes learned expert embeddings and graph message passing to capture dependencies between different sub-benchmarks.

We explicitly contrast these learning-based routers against static heuristics. Strategies such as `greedy_fastest` (always selecting tests with the shortest wall-clock time), `greedy_slowest`, or fixed CPU/I/O mixes fail to adapt to unseen system states. By conditioning selection on real-time telemetry, the router avoids wasting time on globally fast but locally uninformative tests.

End-to-End Evaluation and the Accuracy-Cost Trade-off. Router models are not evaluated in isolation based solely on their training loss. Instead, MoEBench utilizes a $3 \times 2 \times 3$ grid evaluation (combining the 3 Router variants, 2 probe regressors, and 3 Reconstructor models) to assess end-to-end performance. The ultimate selection metric is the final reconstructed suite error (MAE and RMSE) produced by the combined pipeline.

This design frames benchmark acceleration as an explicit accuracy-cost trade-off, managed through two mechanisms:

(1) **Top-K Sweep:** By evaluating configurations from $K = 1$ through full set, the system balances the execution time saved against the reconstruction error. A combined balanced score penalizes configurations that yield high error or fail to deliver meaningful time savings.
(2) **Uncertainty-Guided Active Refinement:** A fixed Top-K budget may be insufficient for highly anomalous systems. After executing the initial subset, the reconstruction model predicts the full-suite score $\hat{y}_{\text{suite}}$ alongside its statistical uncertainty σ_{suite}. If the uncertainty exceeds a predefined threshold ($\sigma_{\text{suite}} \leq \tau_\sigma$), MoEBench dynamically selects the unexecuted expert with the highest predicted variance:

$$e_{\text{next}} = \arg\max_{e_i \notin \mathcal{S}} \sigma_i$$

This extra test is executed, merged into the observation vector, and the reconstruction is invoked again, repeating until confidence is achieved or the execution budget is exhausted.

3.4 Time-Bounded Probing

In addition to reducing the number of executed sub-benchmarks, MoEBench further reduces the evaluation cost of each selected expert through time-bounded probing. Instead of running an iterative sub-benchmark until its official loop completes, MoEBench executes a short probe for a predefined time window and leverages the collected telemetry to estimate the full sub-benchmark score.

This mechanism differs fundamentally from confidence-based adaptive early stopping. The probe duration is strictly clamped within a fixed, safe operational window (e.g., 1 to 30 seconds) to avoid runaway executions caused by anomalous inputs. MoEBench does not repeatedly query a model to decide whether to halt; rather, it enforces a hard wall-clock limit and uses a supervised estimator to map the truncated execution signature to the final benchmark score.

Dual-Mode Probing: Micro and Real. To accommodate diverse benchmark characteristics, MoEBench supports two probing modes:

- **Micro-probe Mode:** The system executes a lightweight, category-aligned synthetic workload (e.g., CPU-intensive, syscall-intensive). This mode avoids invoking the heavy initialization logic of the original benchmark, providing a low-overhead approximation of the targeted subsystem.
- **Real-probe Mode:** MoEBench invokes the original benchmark component but wraps it in a strict system-level timeout (e.g., `timeout <duration_s + margin>`). If the subtest fails to complete within the budget, it is forcibly terminated.

Engineering Robustness over Outlier Rejection. Unlike conventional ML pipelines that aggressively discard truncated executions, timeouts, or noisy runs as statistical outliers, MoEBench embraces these execution anomalies as measurable system states. In production environments, telemetry collection is often imperfect due to permission restrictions or heavy load. MoEBench handles these gracefully:

- **Encoding Truncation as Features:** If a real probe is killed by the timeout, the event is not discarded. Instead, flags such as `real_timed_out`, `workload_rc` (return code), and elapsed wall-clock time are explicitly encoded into the feature vector. The model learns to interpret the telemetry context of a forcefully truncated run.
- **Graceful Degradation for eBPF:** If high-fidelity eBPF tracing is unavailable (due to kernel restrictions or missing tools), the framework does not crash. It flags `ebpf_available = 0` and zero-masks the relevant dimensions, allowing the estimator to infer based solely on standard `/proc` counters.
- **Rate Conversion:** To ensure telemetry is comparable regardless of the exact elapsed probe duration, monotonically increasing metrics (e.g., eBPF `sched_switch` or `syscall_enter`) are converted to per-second rates prior to vectorization.

Label Stabilization and Inference. The probe model is trained using historical full benchmark runs. During training construction, samples lacking valid ground-truth labels from full runs are safely filtered out. For benchmark suites with massive scoring variances across different hardware generations (e.g., Phoronix Test Suite), the primary prediction targets undergo $\log(1+x)$ (log1p) scaling to stabilize the loss gradients. Furthermore, to prevent overfitting on sparse data, the framework enforces a minimum sample threshold for per-test estimators.

At inference time, MoEBench simply executes the bounded probe, processes the robust feature vector, and predicts the scaled full-run score. This design changes the cost of evaluating selected subtests from an unpredictable, unbounded benchmark duration to a fixed and highly controllable probing budget.

3.5 Reconstructor: Full-Suite Result Recovery

After the router selects and executes a small subset of sub-benchmarks, MoEBench relies on a reconstructor to recover the full benchmark outcome. The reconstructor addresses the sparse observation problem: given the current system feature vector and the partial subtest results, it acts as a multi-output regression model to simultaneously predict the scores of all unexecuted sub-benchmarks as well as the aggregate suite score.

Input Representation and Multi-Output Target. To make sparse benchmark results consumable by a fixed-size tensor, MoEBench encodes all sub-benchmarks in a canonical order. Each sub-benchmark is represented by a 3-tuple: $z_i = (m_i, v_i, t_i)$, where $m_i \in \{0, 1\}$ is a binary observation mask, v_i is the observed score (if executed), and t_i is the observed execution time. For unexecuted tests, the tuple is zero-filled. The final input vector concatenates the system context x with the masked partial observations:

$$X = [x,\ m_1, v_1, t_1,\ \ldots,\ m_n, v_n, t_n]$$

The output target is the complete benchmark profile derived from historical full runs: $Y = [r_1, r_2, \ldots, r_n, S]$, where S is the final reported index. For tree-based models (e.g., LightGBM, XGBoost), MoEBench wraps the base regressor with a multi-output strategy to train an independent regressor per target dimension.

Training via Simulated Sparse Execution. Historical datasets exclusively contain complete benchmark runs. To teach the reconstructor how to recover full results from sparse router selections, MoEBench employs a dynamic masking augmentation strategy during training. Each complete historical run is expanded into multiple simulated partial observations. For each augmentation pass, the framework uniformly samples a subset size $k \in [k_{\min}, k_{\max}]$, randomly selects k subtests to act as the "executed" set, and masks the rest. This robust augmentation exposes the model to diverse sparsity patterns, preventing it from over-relying on any single subtest.

Scale Compression for Heterogeneous Targets. Comprehensive suites like the Phoronix Test Suite (PTS) aggregate highly heterogeneous workloads whose primary metrics span drastically different magnitudes (e.g., millions of operations per second vs. submillisecond latencies). To prevent workloads with massive numerical scales from dominating the loss gradients, MoEBench applies domain-aware scale compression. Target variables spanning large orders of magnitude undergo $\log(1 + x)$ transformations, and repeated profile outcomes are aggregated via log-mean. This stabilizes the training process without requiring complex, instance-weighted loss functions.

Uncertainty Estimation over Forced Rebalancing. A common challenge in system telemetry is the long-tail distribution of machine states; certain hardware configurations or severe contention scenarios appear rarely in the training data. Instead of forcefully rebalancing the dataset using synthetic oversampling or focal loss—which risks distorting the natural distribution of system states—MoEBench embraces a system-level fail-safe: uncertainty estimation.

Rather than blindly trusting predictions on rare machine states, MoEBench trains the reconstructor to quantify its own confidence. For MLP implementations, MoEBench utilizes a heteroscedastic network architecture trained via Gaussian Negative Log-Likelihood (NLL) to output both the predicted mean and a log-variance for each subtest. For tree-based models, MoEBench trains an auxiliary residual model on the primary model's absolute training errors (i.e., $|y - \hat{y}|$) to serve as a proxy for variance (σ).

Consequently, if the reconstructor encounters an anomalous, out-of-distribution system state, it will output a high uncertainty variance. As described in Section 3.3, this uncertainty directly triggers the active refinement loop, instructing the system to physically execute more subtests rather than relying on a low-confidence prediction.

Table 2: Evaluation testbeds. Five Linux hosts with heterogeneous CPU and memory capacity. PTS-GPU runs only on H1.

Host	vCPU	RAM (GiB)	GPU	Suites
H1	32	128	NVIDIA	UnixBench, PTS-CPU, PTS-GPU
H2	2	8	—	UnixBench, PTS-CPU
H3	4	8	—	UnixBench, PTS-CPU
H4	4	16	—	UnixBench, PTS-CPU
H5	8	8	—	UnixBench, PTS-CPU

4 EVALUATION

4.1 Experimental Setup

We evaluate MoEBench on five heterogeneous Linux hosts. UnixBench and PTS-CPU are evaluated on all five machines; PTS-GPU is evaluated only on H1 (32U128G), the sole host equipped with an NVIDIA GPU—the remaining four hosts are CPU-only and do not run GPU workloads. Our **primary method and main experiment** is the integrated **Hybrid pipeline**: collect $\mathbf{x}$ once, use a router to select Top-K sub-benchmarks, run a short probe (default 4 s) on *only* those selected components, and reconstruct the full-suite score from the probe-predicted partial observations. Ground-truth full-suite scores and full-suite wall times come from previously collected complete runs on the same host (offline replay); we do *not* re-execute full suites during hybrid evaluation. We exhaustively compare all $3 \times 2 \times 3 = 18$ combinations of three routers, two probe regressors (LightGBM and XGBoost), and three reconstructors under this hybrid protocol on each applicable host–suite pair. **Route A** (Top-K *full* partial execution + reconstruction) and **Route B** (probe *all* components and aggregate without reconstruction) are retained as *ablation baselines* that isolate subset selection and probing, respectively. Unless stated otherwise, each host trains and evaluates only on its own collected runs, so reported results reflect per-machine generalization rather than cross-host pooling. All numeric results in the tables below are left as placeholders and will be filled after the full experimental campaign completes.

4.1.1 Testbeds. We deploy MoEBench on five Linux machines spanning a wide range of CPU and memory capacities. Table 2 summarizes the hardware configuration of each host and the benchmark suites executed on it. H1 (32 logical CPUs, 128 GiB RAM, NVIDIA GPU) serves as the development server and is the only machine on which PTS-GPU is run; H2–H5 are smaller CPU-only instances used for UnixBench and PTS-CPU evaluation. All machines share the same software stack (kernel, compiler toolchain, and benchmark versions) modulo platform-specific GPU drivers on H1. We label hosts by a compact `CPU×RAM` shorthand used throughout this section and in Section 2.

Software environment. Each host uses a Linux kernel with standard `perf` and optional `bpftrace` support where permitted. System feature collection ($\mathbf{x}$) uses a fixed warmup window of 3 s, `/proc` sampling interval of 0.5 s, and a 64 MiB in-process warmup working set unless noted otherwise. When `perf` PMU counters are blocked (e.g., due to `kernel.perf_event_paranoid`), the collector degrades to

`/proc`-based proxies without aborting the pipeline. For GPU workloads on H1, NVIDIA driver and OpenCL inventory are collected when available; on CPU-only hosts (H2–H5), GPU dimensions are zero-masked at inference time.

Per-host training protocol. For every host $h \in \{\text{H1}, \ldots, \text{H5}\}$, we collect benchmark datasets locally and train all models using only runs whose session tags match h. H1 trains and evaluates on UnixBench, PTS-CPU, and PTS-GPU; H2–H5 train and evaluate on UnixBench and PTS-CPU only. This isolates each machine's router, reconstructor, and probe models from other hosts' data distributions. Cross-host comparison therefore reflects how MoEBench behaves under different hardware ceilings rather than a single global model transferred across machines.

4.1.2 Benchmark Suites. We evaluate on three benchmark suites that stress complementary subsystems. UnixBench and PTS-CPU are run on all five hosts; PTS-GPU is restricted to H1 because the other four machines lack discrete GPUs.

- **UnixBench (UB).** A classic system-level suite with 12 single-copy subtests ($N = 12$), including CPU kernels (`dhry2reg`, `whetstone-double`), I/O (`fstime`, `fsbuffer`, `fsdisk`), and OS/interaction workloads (`pipe`, `context1`, `spawn`, `syscall`, `shell1`, `shell8`). We run UnixBench with a single parallel copy (`perl Run -c 1`), matching the MoEBench collection protocol. The aggregate metric is the *System Benchmarks Index Score*. Evaluated on all five hosts.
- **Phoronix Test Suite — CPU (PTS-CPU).** The standard `cpu` suite with approximately 13 profiles covering compilation, cryptography, compression, and numerical kernels. Profile-level primary values span large dynamic ranges; suite aggregation follows log-mean over profiles, consistent with our reconstructor training target. Evaluated on all five hosts.
- **Phoronix Test Suite — GPU (PTS-GPU).** The NVIDIA GPU compute suite (PTS-GPU), executed only on H1 (32U128G), which has a suitable NVIDIA GPU and installed PTS profiles. This suite complements CPU-centric evaluation with OpenCL/GPU compute workloads and is omitted on H2–H5.

For hybrid and Route A end-to-end grid experiments on PTS, ground-truth full-suite scores are taken from previously collected full runs on the same host (offline replay), avoiding repeated full-suite online execution that can cause excessive wall time or resource exhaustion. Hybrid UnixBench evaluation likewise replays historical runs for accuracy and uses dataset-recorded full-suite wall times for cost comparison.

4.1.3 Collected Data.

Phase 1: Full-suite collection (shared by all routes). On each host, we collect multiple complete benchmark sessions (typically 3–5 rounds per session) for every suite applicable to that host (UnixBench and PTS-CPU on all five hosts; PTS-GPU on H1 only). Each run record stores: (i) system context $\mathbf{x}$ (static + dynamic features), (ii) per-component results $\mathbf{y}$ and timings $\mathbf{t}$, and (iii) expert metadata (component identifiers, categories, and titles). These runs provide supervision for router training, reconstructor training, probe label construction, and ground-truth evaluation.

Phase 2: Hybrid main experiment (router + probe + reconstruct). For each applicable suite on each host, we train three router architectures (LightGBM ranker, MLP, GNN-expert), three reconstructors (XGBoost, LightGBM, MLP), and two probe regressors (LightGBM and XGBoost). We evaluate all $3 \times 2 \times 3 = 18$ hybrid combinations offline: for each historical run, the router selects Top-K components, probe predictions from the selected regressor substitute for executed subtest scores on the selected subset only, and the reconstructor predicts the suite score. Partial wall time is estimated as $\hat{T}_{\text{xi}} + K \cdot T_{\text{probe}}$ (default $\hat{T}_{\text{xi}} = 3$ s, $T_{\text{probe}} = 4$ s); full-suite wall time is read from the dataset.

Phase 3: Ablation — Route A only (partial full execution + reconstruction). We repeat the 3×3 router–reconstructor grid using *actual* partial sub-benchmark execution scores (not probe predictions) on the router-selected subset, matching the original Route A design.

Phase 4: Ablation — Route B only (probe all components). For each applicable suite on each host, we build a probe dataset by executing short probes (default 4 s per component, micro-probe mode) over historical full runs on the same host, train two regressors (LightGBM and XGBoost), and evaluate suite prediction by probing *every* component and aggregating—without router selection or reconstruction.

Phase 5: Supplementary offline studies (Hybrid best combination). After selecting the best router–reconstructor pair per suite from Phase 2 on each applicable host, we conduct three supplementary studies on stored runs only (leave-one-session-out cross-validation, no re-execution): Top-K sweep, routing-policy comparison, and system-feature ablation. For PTS-GPU, supplementary studies use H1 data only.

4.1.4 Metrics.

Accuracy. For suite-level score $\hat{S}$ and ground truth S, we report: *absolute error* $|\hat{S} - S|$, *relative error* $|\hat{S} - S|/\max(|S|, \epsilon)$, and rank-correlation metrics (Spearman/Kendall) where cross-run ranking is meaningful. For offline cross-validation studies, we also report out-of-fold MAE and RMSE on the suite dimension.

Cost. We measure *wall-clock time* of benchmark execution: T_{partial} for the executed subset, T_{full} for the complete suite (or dataset-replay equivalent), and *fraction saved* $(T_{\text{full}} - T_{\text{partial}})/T_{\text{full}}$. Route B additionally reports total probe wall time and per-component probe duration.

Combined trade-off score. For Top-K and routing-policy studies, we rank configurations using a *balanced score* that penalizes high suite MAE and low time savings (lower is better), enabling Pareto-style comparison between accuracy and cost.

Aggregation across hosts. For UnixBench and PTS-CPU, we summarize each metric by mean (and optionally standard deviation) over the five testbeds unless a table explicitly reports per-host columns. For PTS-GPU, all reported values come from H1 (32U128G) only. Statistical significance testing will be reported where applicable after all runs complete.

4.2 Overall Effectiveness

We first ask whether MoEBench can approximate full-suite scores with substantially lower execution cost than running every component to completion. For each applicable host–suite pair, we compare: **(i)** full-suite execution (baseline), **(ii)** the **Hybrid pipeline** with the best $3 \times 2 \times 3$ router–probe–reconstructor combination from the main grid on that host, **(iii)** Route A (ablation: partial *full* execution + reconstruction), and **(iv)** Route B (ablation: probe all components without reconstruction).

Table 3 reports suite-level relative error, partial and full wall times, and time saved. UnixBench and PTS-CPU rows aggregate over five hosts; PTS-GPU rows report H1 (32U128G) only. We expect Hybrid to combine Route A's subset efficiency with Route B's low per-component probe cost, outperforming both ablations on the accuracy–cost Pareto frontier.

4.3 Model Combination Study

The Hybrid pipeline depends on the router, the probe regressor, and the reconstructor. We exhaustively evaluate the Cartesian product of three routers {LightGBM, MLP, GNN}, two probe regressors {LightGBM, XGBoost}, and three reconstructors {XGBoost, LightGBM, MLP}, yielding eighteen combinations per applicable host–suite pair under the hybrid protocol (router Top-K → probe on selected components → reconstruct). Table 4 shows UnixBench results (mean over five hosts); Table 5 shows PTS-CPU (mean over five hosts); Table 6 shows PTS-GPU on H1 only. Each cell will report mean suite relative error and mean hybrid wall time ($\hat{T}_{\text{xi}} + K \cdot T_{\text{probe}}$) under the corresponding aggregation scope, aggregated over both probe regressors unless a table explicitly fixes one. The Route A-only 3×3 grid (partial full execution instead of probe predictions) is reported separately as an ablation in supplementary material.

We will identify the *best combination* per suite (lowest suite error subject to meaningful time savings) and use its router checkpoint for the supplementary studies in Sections 4.4–4.6.

4.4 Accuracy–Cost Trade-off

Given the best router from Section 4.3, we sweep the subset size $K \in \{1, 2, 3, 4, 5, 6\}$ (capped by the number of components in each suite). For each K, the router selects Top-K experts, partial results are observed, and the reconstructor predicts the suite score. Evaluation uses leave-one-session-out cross-validation on each host's historical runs, with the router providing subset selection at inference time. UnixBench and PTS-CPU results are aggregated over five hosts; PTS-GPU results are from H1 only.

Table 7 reports, for each suite, suite MAE, mean wall-time saved fraction, and balanced score at each K. The recommended $K^\star$ minimizes balanced score while keeping relative error below a suite-specific tolerance (to be fixed after observing the error scale).

Recommended values $K^\star_{\text{UB}}$, $K^\star_{\text{CPU}}$, and $K^\star_{\text{GPU}}$ will be used as fixed subset sizes in Section 4.5.

4.5 Routing Strategy Comparison

Subset selection strategy strongly influences both cost and reconstruction quality. At fixed $K = K^\star$ from Section 4.4, we compare seven policies: `random` (seeded uniform subset), `fixed_first_k` (canonical order prefix), `fixed_cpu_mix` and `fixed_io_mix` (heuristic CPU/IO presets; on PTS, degenerates to canonical prefix when IDs do not match), `greedy_slowest` and `greedy_fastest` (by historical wall time on the held-out sample), and `router` (Top-K from the trained router). All policies share the same reconstructor and cross-validation protocol. UnixBench and PTS-CPU results are aggregated over five hosts; PTS-GPU results are from H1 only.

Table 8 summarizes suite MAE, time saved, and balanced score. We hypothesize that `router` achieves the best trade-off by conditioning on $\mathbf{x}$, while static heuristics fail on heterogeneous hosts (e.g., I/O-bound vs. CPU-bound configurations).

4.6 Feature Ablation Study

To quantify which parts of the system context $\mathbf{x}$ drive reconstruction accuracy, we ablate feature groups while holding the router policy and $K^\star$ fixed. We evaluate five modes: **full** (complete $\mathbf{x}$), **static_hw_only** (hardware topology, memory, cache, GPU inventory; zero dynamic/PMU), **no_perf_pmu** (remove PMU-derived rates except `perf_event_paranoid`), **no_dynamic_proc** (remove load, fault, and bandwidth-proxy dynamics), and **no_gpu** (remove GPU/OpenCL dimensions). For each mode, we report suite MAE and the *MAE increase relative to full* ΔMAE; larger Δ indicates higher importance of the removed group. UnixBench and PTS-CPU results are aggregated over five hosts; PTS-GPU results are from H1 only.

Table 9 lists results per suite under the corresponding aggregation scope. The importance ranking will guide which telemetry to prioritize in production deployments with restricted permissions.

4.7 Probe-Based Evaluation (Ablation)

Route B (ablation) replaces long sub-benchmark execution with time-bounded probing (default 4 s per component, micro-probe mode with optional eBPF). Unlike the Hybrid pipeline, Route B probes *every* component and aggregates predicted scores directly—without router subset selection or reconstruction. We train two probe regressors—LightGBM and XGBoost—on probe features $\mathbf{p}_{i,j}$ collected from each host's historical full runs, then evaluate online suite prediction against the latest full run on the same host. For PTS suites, labels use $\log(1 + x)$ scaling with per-profile estimators to handle heterogeneous metric magnitudes. Probe evaluation covers UnixBench and PTS-CPU on all five hosts and PTS-GPU on H1 only (11 host–suite pairs in total).

Table **??** compares LightGBM vs. XGBoost on suite relative error and total probe wall time. UnixBench and PTS-CPU rows aggregate over five hosts; the PTS-GPU row reports H1 only. Table **??** provides a per-host breakdown for UnixBench (all five hosts).

4.8 Summary of Findings

Our evaluation on five heterogeneous Linux hosts is designed to answer four questions. We run UnixBench and PTS-CPU on all five machines and PTS-GPU on H1 only. **Q1:** Can the Hybrid pipeline approximate full-suite scores with significantly less execution time than the baseline? **Q2:** Which router–reconstructor combination generalizes best per suite under Hybrid evaluation? **Q3:** How should practitioners choose K, subset policy, and telemetry groups under

Table 3: Overall effectiveness. UnixBench and PTS-CPU: mean over five hosts. PTS-GPU: H1 (32U128G) only. Best Hybrid, Route A, and Route B configurations are selected per host. Numeric cells are placeholders.

Suite	Method	Rel. err. (%)	T_{partial} (s)	T_{full} (s)	Saved (%)	MAE	Spearman
UnixBench	Full suite	0	—	—	0	—	—
	Hybrid (main)	—	—	—	—	—	—
	Route A (abl.)	—	—	—	—	—	—
	Route B (abl.)	—	—	—	—	—	—
PTS-CPU	Full suite	0	—	—	0	—	—
	Hybrid (main)	—	—	—	—	—	—
	Route A (abl.)	—	—	—	—	—	—
	Route B (abl.)	—	—	—	—	—	—
PTS-GPU	Full suite	0	—	—	0	—	—
	Hybrid (main)	—	—	—	—	—	—
	Route A (abl.)	—	—	—	—	—	—
	Route B (abl.)	—	—	—	—	—	—

Table 4: Hybrid model combination study on UnixBench (main experiment). Rows: routers; columns: reconstructors. Values: mean suite relative error (%) / mean hybrid wall time (s) over five hosts. Placeholders.

Router \ Reconstructor	XGBoost	LightGBM	MLP
LightGBM	— / —	— / —	— / —
MLP	— / —	— / —	— / —
GNN-expert	— / —	— / —	— / —

Table 5: Hybrid model combination on PTS-CPU (mean over five hosts). Placeholders.

Router \ Recon.	XGB	LGBM	MLP
LightGBM	—	—	—
MLP	—	—	—
GNN	—	—	—

Table 6: Hybrid model combination on PTS-GPU (H1 / 32U128G only). Placeholders.

Router \ Recon.	XGB	LGBM	MLP
LightGBM	—	—	—
MLP	—	—	—
GNN	—	—	—

an accuracy–cost budget? **Q4:** How do Route A and Route B ablations compare to Hybrid, and which probe regressor is preferable for Route B?

After all experiments complete, we will summarize findings as follows (placeholders to be replaced with measured conclusions):

- **Overall effectiveness (Q1).** Hybrid reduces UnixBench wall time by **[TBD]%** on average with **[TBD]%** mean relative suite error, outperforming Route A and Route B ablations on the accuracy–cost frontier (Table 3).
- **Model combinations (Q2).** The best Hybrid pair per suite is **[TBD router]** + **[TBD reconstructor]** for UnixBench (five hosts), **[TBD]** for PTS-CPU (five hosts), and **[TBD]** for PTS-GPU (H1 only) (Tables 4–6).
- **Accuracy–cost trade-off (Q3a).** Recommended subset sizes are $K^{\star}_{\text{UB}}$ = **[TBD]**, $K^{\star}_{\text{CPU}}$ = **[TBD]**, and $K^{\star}_{\text{GPU}}$ = **[TBD]** on H1 (Table 7).
- **Routing policies (Q3b).** The learned `router` policy outperforms static heuristics by **[TBD]** balanced-score margin on average for UnixBench and PTS-CPU across five hosts, and on H1 for PTS-GPU (Table 8).
- **Feature importance (Q3c).** Dynamic PMU/proc features contribute **[TBD]** ΔMAE when removed; GPU/OpenCL features matter most for PTS-GPU on H1 (Table 9).
- **Ablations (Q4).** Route A and Route B each sacrifice accuracy or cost relative to Hybrid; **[LightGBM / XGBoost]** yields lower Route B suite error on **[TBD]** of 11 host–suite pairs (Table **??**).

These results collectively demonstrate that MoEBench makes full-suite benchmarking practical in settings where repeated, low-disruption measurement is required—including multi-machine comparison, configuration tuning, and production-adjacent monitoring—without abandoning the interpretability of established suite scores.

REFERENCES

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

Table 7: Top-K sweep using the best router–reconstructor pair per suite (offline CV). UnixBench and PTS-CPU: mean over five hosts. PTS-GPU: H1 only. Bal. = balanced score (lower is better). Placeholders.

Suite	K	MAE	RMSE	Spearman	Saved (%)	Bal.	Rank
UnixBench	1	—	—	—	—	—	—
	2	—	—	—	—	—	—
	3	—	—	—	—	—	—
	4	—	—	—	—	—	—
	5	—	—	—	—	—	—
	6	—	—	—	—	—	—
PTS-CPU	1	—	—	—	—	—	—
	2	—	—	—	—	—	—
	3	—	—	—	—	—	—
	4	—	—	—	—	—	—
	5	—	—	—	—	—	—
	6	—	—	—	—	—	—
PTS-GPU	1	—	—	—	—	—	—
	2	—	—	—	—	—	—
	3	—	—	—	—	—	—
	4	—	—	—	—	—	—
	5	—	—	—	—	—	—
	6	—	—	—	—	—	—

Table 8: Routing / subset policy comparison at fixed $K^\star$ (offline CV). UnixBench and PTS-CPU: mean over five hosts. PTS-GPU: H1 only. Placeholders.

Suite	Policy	MAE	Saved (%)	Bal.	Rank
UnixBench	random	—	—	—	—
	fixed_first_k	—	—	—	—
	fixed_cpu_mix	—	—	—	—
	fixed_io_mix	—	—	—	—
	greedy_slowest	—	—	—	—
	greedy_fastest	—	—	—	—
	router	—	—	—	—
PTS-CPU	random	—	—	—	—
	fixed_first_k	—	—	—	—
	fixed_cpu_mix	—	—	—	—
	fixed_io_mix	—	—	—	—
	greedy_slowest	—	—	—	—
	greedy_fastest	—	—	—	—
	router	—	—	—	—
PTS-GPU	random	—	—	—	—
	fixed_first_k	—	—	—	—
	fixed_cpu_mix	—	—	—	—
	fixed_io_mix	—	—	—	—
	greedy_slowest	—	—	—	—
	greedy_fastest	—	—	—	—
	router	—	—	—	—

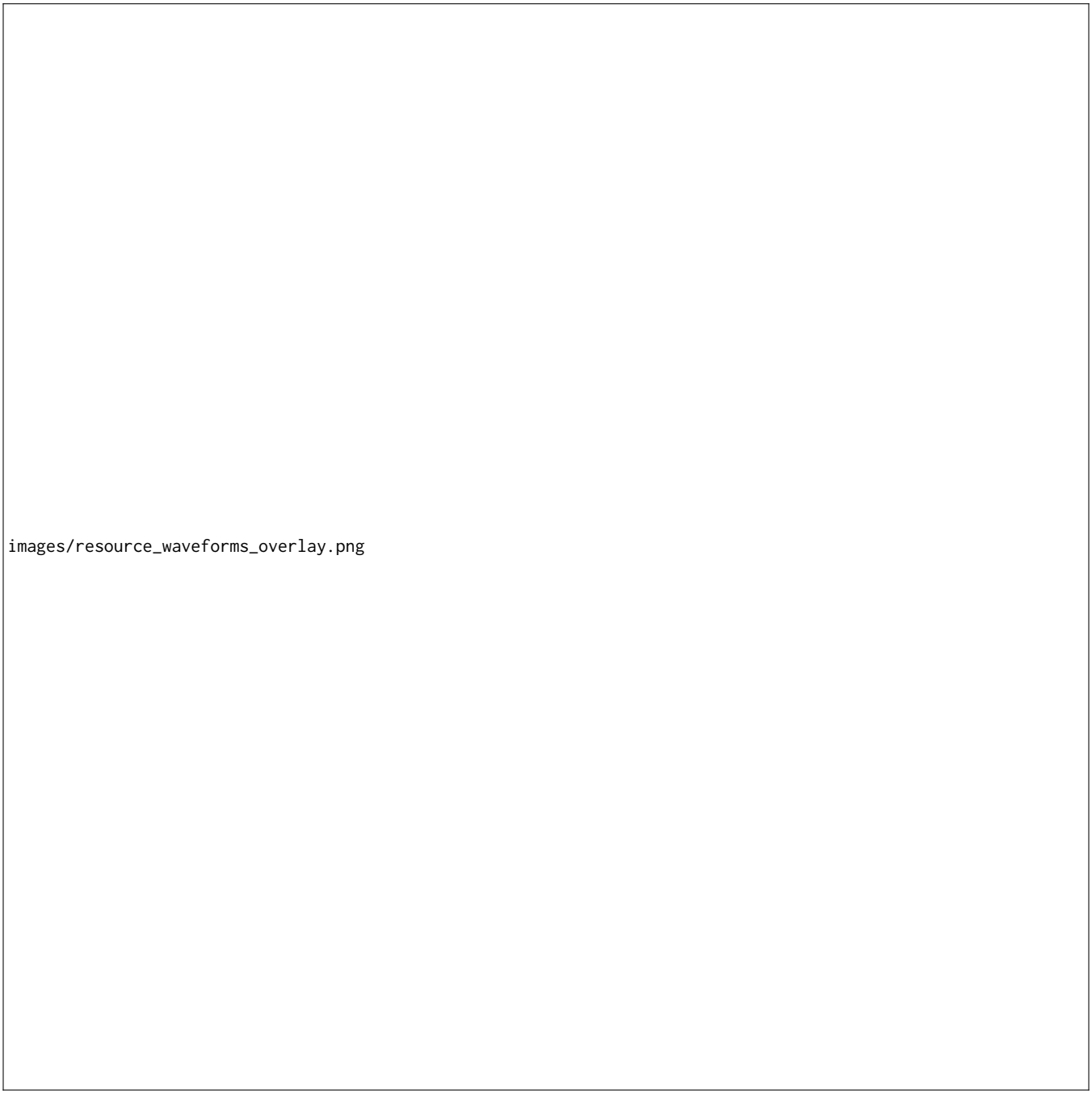


Figure 3: Enter Caption

Table 9: System-feature ablation at fixed router policy and $K^\star$ (offline CV). UnixBench and PTS-CPU: mean over five hosts. PTS-GPU: H1 only. ΔMAE is increase vs. `full`. Placeholders.

Suite	Ablation mode	MAE	ΔMAE	Importance rank
UnixBench	full	—	0	—
	static_hw_only	—	—	—
	no_perf_pmu	—	—	—
	no_dynamic_proc	—	—	—
	no_gpu	—	—	—
PTS-CPU	full	—	0	—
	static_hw_only	—	—	—
	no_perf_pmu	—	—	—
	no_dynamic_proc	—	—	—
	no_gpu	—	—	—
PTS-GPU	full	—	0	—
	static_hw_only	—	—	—
	no_perf_pmu	—	—	—
	no_dynamic_proc	—	—	—
	no_gpu	—	—	—